NILM Project - Milestone 2

Modeling, Simulation and Automation of Electrical Energy Systems

Team	Soheil Ayati (11153003), Marc Steffgen (11149043)
Milestone	3 – Live NILM, On-The-Go Training
Repository	https://github.com/SoheilAyati/MSAEES

1. Objectives

Milestone 3 turns the offline NILM pipeline from Milestone 2 into a running laboratory system. The project can now connect to the Siemens PAC4200, read the live aggregate signal, disaggregate it into per-device estimates, show what is currently on, and record exact switching events. The dashboard also answers the transfer question from the milestone brief: the output is no longer only a model score or a CSV file, but a live operator-facing estimate of devices, watts, confidence, residual power, and event history.

A second goal was robustness. Instead of forcing every watt into a known class, the system monitors unexplained residual power. Sustained residuals trigger an unknown-device workflow: the user names the device, the system guides a clean recording, rebuilds measured scenarios, retrains the mix and identify models in the background, and hot-reloads the new bundles. This makes the appliance vocabulary dynamic rather than hard-coded.

2. Live pipeline architecture

The Milestone 3 live monitor is implemented in Scripts/MS2_Pipeline/live.py and builds on the PAC4200 reader from Milestone 2. A persistent acquisition service polls the meter at about 5 Hz, keeps a rolling buffer, and provides the same channels that the offline pipeline expects. Every few seconds the LiveEngine converts the trailing window into the aggregate feature row used during training, evaluates the mix bundle, and publishes the current device state through a browser dashboard.

- **Current state.** For each detected device the dashboard shows watts, confidence, and the time since the device was switched on.
- **Event log.** A step detector runs on P and Q and logs edge_on and edge_off events with millisecond timestamps, delta P, delta Q, confidence, and the matched device.
- **Trust signal.** The UI displays model provenance, held-out metrics, and an explained-power fraction so the user can see when the current decomposition is complete.
- **Replay mode.** Measured HDF5 or CSV recordings can be replayed through the same live path. This allows the recognition, edge detection, and teach loop to be tested without the PAC4200 being physically connected.

3. Data and feature updates

The feature basis was revised before the live validation. The current mix model uses 17 aggregate features rather than the earlier steady-state-only set. The first group captures active and reactive level, phase localisation, power factor, apparent power, and THD. The added event features are Pstep_max, Qstep_at_Pstep, and n_steps. They encode the largest settled switching step inside the window and therefore inject Hart-style transient information into a window model.

This change matters because steady-state sums alone can confuse a new device with a power drift of an already active device. A boiler plus a lamp can look like a boiler drawing more power, but the switch-on edge has its own delta P and delta Q signature. In the live engine, matched edges therefore claim the device state directly: an on-edge claims a device with the step watts, an off-edge releases it, and the remaining model-estimated watts are rescaled to the measured total. A physical guard drops stale claims if the claimed power exceeds the measured aggregate.

The harmonic path was also corrected and integrated. The PAC4200 does not expose current harmonics as a normal register block on this installation; they are read via Modbus function code 0x14 file records. Current harmonic file 113 on L1 was verified against a real load, and recordings now store orders 2 through 40 at the full 5 Hz rate. The meter provides magnitudes but not harmonic phases, so real recordings mark

harmonic_phase_captured as false. The live system derives THD_I from the same per-order spectrum used by the measured-scenario aggregator, keeping training and deployment features aligned.

The data basis considered by the tabular model path was updated accordingly. Random Forest and LightGBM now operate on the same physically interpretable feature matrix, including the harmonic summaries h3, h5, h7, h_centroid, and h_energy where per-order current spectra are available. The full 39-bin spectrum remains stored in the HDF5 files, but the model input uses compact summaries to avoid overfitting on the current small measured corpus. This keeps the LightGBM comparison meaningful: differences between RF and LGBM reflect the model family, not different input data.

4. Recognition and training on the go

Recognition combines the mix model with event-state logic. Presence probabilities are median-smoothed over recent strides and passed through hysteresis so borderline windows do not flicker. Edges are detected from settled medians before and after a jump, debounced, and matched against single-device signatures. The event timestamp is then used as the device's switching time, so the dashboard can report when the device changed rather than only when the next inference window finished.

Unknown devices are handled through the residual. If the measured power is not sufficiently explained for at least eight seconds, the dashboard prompts for a device name. The final implementation uses a guided isolated recording instead of in-mix subtraction, because clean isolated captures produced more stable retraining data. The guided flow asks the user to disconnect all devices, records an off baseline, waits for only the new device, records its steady operation, then records an off tail. The new HDF5 recording is mixed into fresh measured scenarios, the mix and identify models are retrained, and the model reloads without restarting the live session.

After a model reload, the engine re-matches all recorded edges from the current session against the new signature table and rebuilds the claims. This is important for transfer: an edge that was previously unrecognized can resolve to the newly taught device after retraining, and the user does not have to physically repeat the whole switching sequence.

5. Results

The current measured-data model is trained on 1920 windows from measured scenarios with a 10 s window and a 5 W on-threshold. The model vocabulary contains coffee_machine, laptop, pv, standing_fan, standing_lamp, table_fan, and water_boiler. Held-out performance improved compared with the frozen original mix model after the data and feature updates.

Model / test	Main result	Interpretation
Original mix bundle	presence F1 0.857, gated MAE 9.5 W	six-device snapshot before the latest measured-data update
Current mix bundle	presence F1 0.916, gated MAE 2.9 W	seven-device vocabulary with event features and updated corpus
Identify bundle	hold-out macro-F1 0.955	single-device recognition with common plus harmonic features
Teach loop validation	unknown after 8 s, retrain in 26 s, then 0.95 confidence	closed loop from unknown load to recognized device

The live tests confirm the same behaviour in the dashboard. Small loads are recognized with a visible residual rather than hidden over-allocation. The expanded model can include newly recorded devices such as a laptop and still show an unrecognized edge when a step matches no known signature. In a high load case, the system split about 1.44 kW into water_boiler, standing_lamp, and table_fan with 99 percent explained power in the dashboard snapshot.

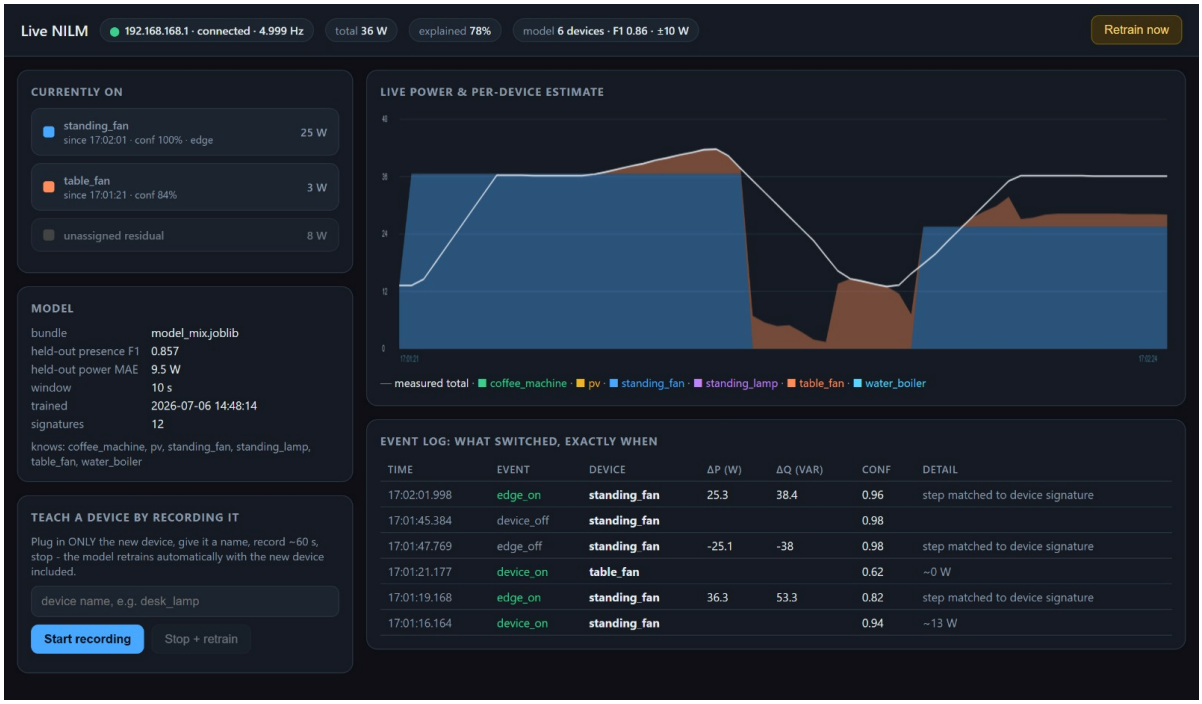

Figure 1. Low-power live split: standing_fan and table_fan are shown with watts, confidence, exact edge events, and a visible residual instead of hiding unexplained power.

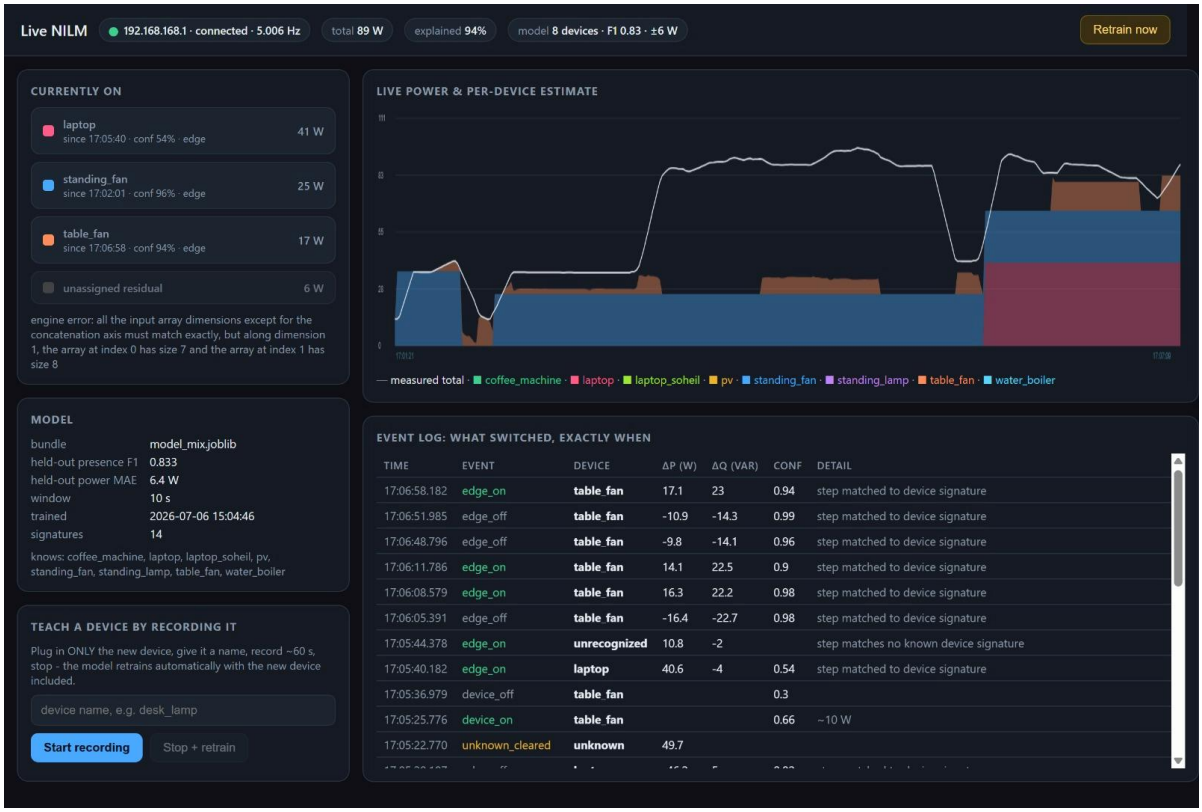

Figure 2. Extended live model after adding devices: the dashboard shows an eight-device vocabulary, laptop recognition, table_fan edge events, and an unrecognized event when no known signature matches.

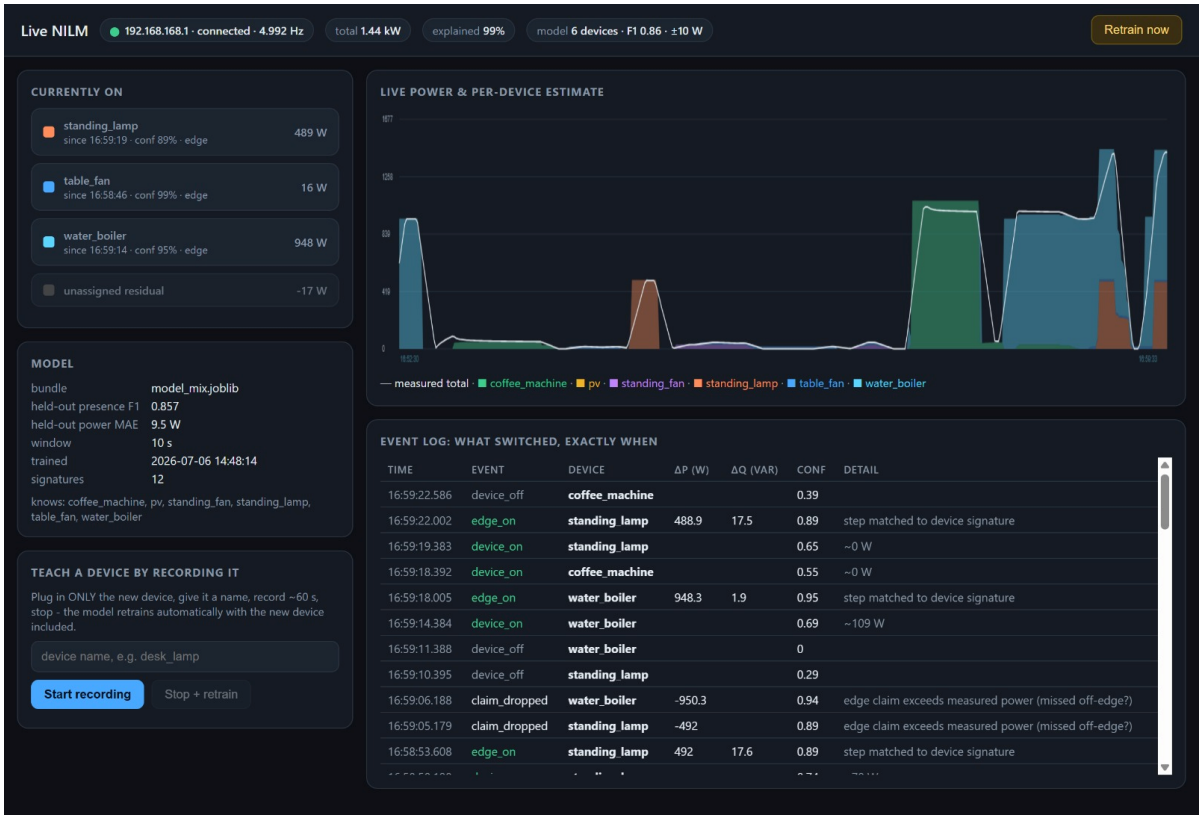

Figure 3. High-load multi-device case: about 1.44 kW is split into water_boiler, standing_lamp, and table_fan with 99 percent explained power in the live dashboard.

6. Transfer and next steps

Milestone 3 changes the practical meaning of the NILM output. The output can now be used as a live laboratory interface: which devices are on, how much each draws, when each switching event happened, how much power remains unexplained, and whether the model should learn a new device. The same outputs are persisted as events.csv and timeline CSV/JSON summaries, so the live run can also be used for later evaluation and paper figures.

The remaining limitations are now specific and testable. The PV recording still contains almost no actual generation, so real PV disaggregation is not yet validated. The two small fans have overlapping P/Q signatures and need more variant recordings. Harmonic phases remain synthetic-only because the PAC4200 does not expose them over Modbus. Finally, the current neural path is still an MLP baseline; longer temporal models such as CNN or LSTM architectures are the natural next step for appliances with multi-minute state cycles.

7. Documentation and repository

The Milestone 3 implementation is documented in Docs/08_live_nilm.md and Docs/09_design_rationale.md. The executable components are live.py, app.py, train.py, infer.py, mix_measured_scenarios.py, and pac_reader.py. The tracked output folder contains the current model bundles and metrics; live session folders are intentionally gitignored because they are per-run logs.